DATA STRUCTURE REMAINING TASKS

MUHAMMAD TAHA
22F-3277
BCS-3C

LAB 4 PROBLEM 3 (OPTIONAL)

```cpp
//Students Enrollment System using Doubly Linked List
#include <iostream>
#include <string>
using namespace std;
struct node
{
    int id;
    string name;
    node* prev;
    node* next;
    node* courseList;  //list for enrolled courses

    //default and ooverloaded constructor
    node(int id = 0, string name = "", node* p = nullptr, node* n = nullptr, node* c =
nullptr) : id(id), name(name), prev(p), next(n), courseList(c) {}
};

class EnrollmentSystem
{
private:
    node* head;
    node* tail;

public:
    EnrollmentSystem() : head (nullptr), tail(nullptr) {}

    //function to enroll a new node
    void enrollStudent(int id, string studentName)
    {
        node* newStudent = new node(id, studentName);

        //if list is empty
        if (head == nullptr)
        {
            head = newStudent;
            tail = newStudent;
        }
        else
        {
            tail->next = newStudent;
            newStudent->prev = tail;
            tail = newStudent;
        }
    }

    //register courses
    void registerCourse(int id, int courseId, string courseName)
    {
        node* current = head;
        while (current != nullptr)
        {
            if (current->id == id)
            {
                node* newCourse = new node;
                //adding new course
                newCourse->id = courseId;
                newCourse->name = courseName;
                //adding previous list after new course
                newCourse->next = current->courseList;
                //updating the list starting
                current->courseList = newCourse;
                break;
            }
            current = current->next;
```

```
    }
}

//unregister courses
void unregisterCourse(int id, int courseId)
{
    node* current = head;
    //this loop runs for student's ;ist
    while (current != nullptr)
    {
        if (current->id == id)
        {
            node* currCourse = current->courseList;

            //this loop runs for courses list of every student
            while (currCourse != nullptr)
            {
                if (currCourse->id == courseId)
                {
                    //if the course to delete is not first in list
                    if (currCourse->prev != nullptr)
                    {
                        currCourse->prev->next = currCourse->next;
                        if (currCourse->next != nullptr)
                        {
                            currCourse->next->prev = currCourse->prev;
                        }

                    }
                    else
                    {
                        current->courseList = currCourse->next;
                        if (currCourse->next != nullptr)
                        {
                            currCourse->next->prev = nullptr;
                        }
                    }

                }
                currCourse = currCourse->next;
            }
            break;
        }
        current = current->next;
    }
}

//print all students
void listAllStudents()
{
    node* current = head;
    while (current != nullptr)
    {
        cout << "ID: " << current->id;
        cout << "\nName: " << current->name;
        cout << "\nCourses Enrolled: ";
        node* courseNode = current->courseList;
        while (courseNode != nullptr)
        {
            cout << courseNode->id << " - ";
            cout << courseNode->name << ", ";
            courseNode = courseNode->next;
        }
        cout << endl << endl;
        current = current->next;
    }
}
```

```
    //search for a node by ID or name
    void searchForStudent(int searchID, string searchName)
    {
        node* current = head;
        while (current != nullptr)
        {
            //search
            if (current->id == searchID || current->name == searchName)
            {
                cout << "ID: " << current->id;
                cout << "\nName: " << current->name;
                cout << "\nCourses Enrolled: ";
                //print whole course list
                node* courseNode = current->courseList;
                while (courseNode != nullptr)
                {
                    cout << courseNode->id << " - ";
                    cout << courseNode->name << ", ";
                    courseNode = courseNode->next;
                }
                cout << endl;
                return;
            }
            current = current->next;
        }
        cout << "Node not found." << endl;
    }
};

int main()
{
    EnrollmentSystem enrollmentSystem;

    enrollmentSystem.enrollStudent(1, "Taha");
    enrollmentSystem.enrollStudent(2, "Sharjeel");
    enrollmentSystem.enrollStudent(3, "Asad");

    enrollmentSystem.registerCourse(1, 2003,"DATA");
    enrollmentSystem.registerCourse(1, 2005,"COAL");
    enrollmentSystem.registerCourse(2, 2005,"COAL");
    enrollmentSystem.registerCourse(2, 1000,"AI");

    enrollmentSystem.registerCourse(3, 2000,"OOP");
    enrollmentSystem.registerCourse(3, 2010,"CALCULUS");

    enrollmentSystem.listAllStudents();
    cout << "\n----------------------------------------\n";

    enrollmentSystem.unregisterCourse(3, 2010);
    cout << "CALCULUS Unregistered\n";
    enrollmentSystem.searchForStudent(3, "");   //searching by ID
    cout << "\n----------------------------------------\n";

    enrollmentSystem.unregisterCourse(1, 2005);
    cout << "COAL Unregistered\n";
    enrollmentSystem.searchForStudent(0, "Taha");   //searching by name

    cout << "\n----------------------------------------\n";

    enrollmentSystem.listAllStudents();


    return 0;
}
```

```
ID: 1
Name: Taha
Courses Enrolled: 2005 - COAL, 2003 - DATA,

ID: 2
Name: Sharjeel
Courses Enrolled: 1000 - AI, 2005 - COAL,

ID: 3
Name: Asad
Courses Enrolled: 2010 - CALCULUS, 2000 - OOP,


----------------------------------------
CALCULUS Unregistered
ID: 3
Name: Asad
Courses Enrolled: 2000 - OOP,

----------------------------------------
COAL Unregistered
ID: 1
Name: Taha
Courses Enrolled: 2003 - DATA,

----------------------------------------
ID: 1
Name: Taha
Courses Enrolled: 2003 - DATA,

ID: 2
Name: Sharjeel
Courses Enrolled: 1000 - AI, 2005 - COAL,

ID: 3
Name: Asad
Courses Enrolled: 2000 - OOP,
```